CARS-SF / RuralSliceAI

Interview Question Bank

Design Choices, Architecture, Core Systems, and 5G Modules

M.Tech Major Project Preparation

August 2026

Contents

1 Project Vision and Research Motivation 2

2 System Architecture and Module Decomposition 2

3 Design Choices and Architectural Trade-offs 3

4 Architectural Correctness and Validation 3

5 5G Fundamentals and Network Slicing 4

6 Private 5G and Rural Deployment Context 4

7 Mininet, OVS, and Network Emulation 5

8 Telemetry, Feature Extraction, and Dataset Design 5

9 SliceNet and Traffic Classification 6

10 Context Engine, Allocation, and QoS Enforcement 6

11 RL Scheduler and Control Loop 7

12 Monitoring, Self-Healing, and Resilience 7

13 Core Systems and Software Engineering 8

14 SDN, OpenFlow, and Linux Networking Internals 8

15 Experimental Methodology and Evaluation 9

16 Scenario-Based and Viva Questions 9

17 Quick-Fire Definitions (Oral Exam) 10

18 Repository Map (Answer Key Reference) 10

1 Project Vision and Research Motivation

1. What is the core research problem your project addresses, and why is traditional eMBB/URLLC/mMTC slicing insufficient for rural deployments?
2. Explain the shift from *technology-centric* slices to *service-centric* slices (Healthcare, Education, Agriculture, General Internet).
3. What is the novelty of CARS-SF if SliceNet and RL already exist independently?
4. Why focus on *rural* private 5G rather than urban macro networks?
5. How does context-awareness (school hours, harvest season, disaster) change allocation compared to static slicing?
6. What societal impact does service-aware slicing claim beyond throughput optimization?
7. Why is bandwidth scarcity a first-class constraint in your architecture rather than an afterthought?
8. How would you defend the project contribution in a viva if asked "Is this just a classifier plus a scheduler?"
9. What future service classes (Emergency, Government, Industrial IoT) would require architectural changes vs. registry extensions?
10. Why is the *complete closed loop* (perceive → decide → act → measure → learn) essential to your thesis claim?

2 System Architecture and Module Decomposition

1. Draw the end-to-end pipeline from incoming traffic to policy improvement. Label every module in your repository.
2. Why separate **perception** (SliceNet) from **control** (RL scheduler)? What breaks if you merge them?
3. Explain the role of `core/orchestrator.py` and why a single entry point matters for reproducibility.
4. What is the purpose of `registries/` (applications, scenarios, features YAML)? Why not hardcode profiles in Python?
5. Why did you preserve `slicing/` (context, priority, allocation) while building new modules around it?
6. What is the difference between `scenario/engine.py` and `context/engine.py`?
7. Why is `traffic_models/demand_generator.py` separate from `traffic/orchestrator.py`?
8. Explain modular interfaces in `core/interfaces.py`: `TelemetrySource`, `FeatureExtractor`, `Labeler`.
9. Why deprecate `MininetSims/` and `RSNSDv2/` instead of deleting them immediately?
10. What is the deployment mapping from Mininet modules to edge server services (`deployment/edge_config.y`
11. How does your architecture support replacing Mininet with a real UPF without rewriting SliceNet?

12. What is the dependency graph between modules? Which modules can run without Mininet?

3 Design Choices and Architectural Trade-offs

1. Why Mininet-first development instead of starting with Open5GS + UERANSIM?
2. Why use Linux `tc` HTB queues instead of only OpenFlow for QoS enforcement?
3. Why registry-driven YAML configuration instead of a central database or protobuf schemas?
4. Why freeze RSNSD v1 synthetic generation rather than continuously extending it?
5. What are the trade-offs of using RandomForest for SliceNet vs. a 1D-CNN as described in literature?
6. Why two-pass weighted fair allocation with surplus redistribution instead of strict priority queuing?
7. Why exclude categorical context columns (`device_type`, `mobility_state`) from ML inputs?
8. Why map Mininet flows to an 89-column RSNSD schema instead of defining a new minimal schema?
9. What are the risks of label leakage when `service_type` appears as both feature and label?
10. Why poll OVS, `tc`, PCAP, and `psutil` synchronously per tick instead of async streaming?
11. Why use spawn metadata for ground-truth labels instead of only post-hoc port heuristics?
12. What breaks if telemetry collectors return stubbed constants (as in early RSNSDv2)?
13. Why is self-healing modular and rule-based first before RL-based recovery?
14. How do you justify synthetic-to-real domain gap when baseline F1 $\gg$ Mininet F1?
15. What would you change if the bottleneck link were 10 Mbps instead of 100 Mbps?

4 Architectural Correctness and Validation

1. How do you verify that allocation decisions actually affect measured QoS in Mininet?
2. What does "architecturally correct" mean for a closed-loop control system in your project?
3. How does `validation/validator.py` ensure dataset consistency? What is the pass threshold?
4. What correlation checks prove internal QoS consistency (e.g., load $\uparrow \Rightarrow$ latency $\uparrow$)?
5. How do you detect when the pipeline produces fake data vs. measured telemetry?
6. What is the ground truth for `service_class` labels in Mininet vs. RSNSD v1?
7. How do you validate that HTB filters correctly map UE IPs to service classes?
8. What test cases in `tests/` cover allocation, monitoring, and scheduler correctness?
9. How would you prove SLA violations are detected within 5 seconds after fault injection?
10. What happens to architectural correctness if PCAP capture fails but iperf3 traffic runs?

11. How do you ensure RL observations originate from measurable telemetry (Markovian state)?
12. What is the traceability matrix (`docs/report_traceability.md`) and why does it matter for viva?
13. How do you reconcile documentation drift (tracker vs. traceability matrix)?
14. What are the failure modes when static routes are used instead of a full SDN controller?
15. How would you audit whether every feature in the 89-column schema has documented provenance?

5 5G Fundamentals and Network Slicing

1. What is network slicing in 5G, and how does it differ from VLANs or DiffServ?
2. Explain eMBB, URLLC, and mMTC. Which service classes in your project map to each?
3. What is a network slice instance (NSI) vs. a network slice subnet (NSS)?
4. What role does the SMF play in slice selection, and what is your project's analogue?
5. What is the UPF, and how does `srv_edge` / `srv_core` approximate it in Mininet?
6. Explain RAN, fronthaul, midhaul, and backhaul in your three-tier topology.
7. What is MEC (Multi-access Edge Computing), and why do you have `mec_hlt` and `mec_ctv` hosts?
8. What are RSRP, RSRQ, SINR, and CQI? Which are measured vs. context-injected in your dataset?
9. What is SLA in 5G slicing, and how is `sla_satisfied` computed in your pipeline?
10. What is slice isolation, and how do HTB queues + u32 filters provide it?
11. How does your project relate to 3GPP service-based architecture (SBA)?
12. What is the difference between hard slicing and soft slicing in your implementation?
13. Why might URLLC healthcare traffic be pinned to the edge server while education goes to core?
14. What is network function virtualization (NFV) relevance to your edge deployment vision?
15. How would Open5GS integration change your architecture without invalidating the core design?

6 Private 5G and Rural Deployment Context

1. What distinguishes a private 5G network from public operator 5G?
2. Why are rural deployments characterized by scarce backhaul and intermittent demand patterns?
3. What is the digital divide argument in your problem statement?
4. How does school-hours context change education slice priority mathematically in `context_engine.py`?

5. What happens to allocation during disaster context? Walk through the two-pass algorithm.
6. Why throttle general internet during medical emergency rather than drop healthcare packets?
7. What is the role of an edge compute server in a rural private 5G deployment?
8. How does fiber cut scenario differ from congestion in terms of telemetry signatures?
9. What regulatory or spectrum considerations affect private 5G in India (if applicable to your thesis)?
10. How would you scale from 4 UEs in Mininet to hundreds of rural users?

7 Mininet, OVS, and Network Emulation

1. Why Mininet for all experiments instead of pure simulation (ns-3) or pure analytics?
2. Explain `topology/rural_topology.py`: hosts, switches, link bandwidths, and delays.
3. Why use `TCLink` instead of default Mininet links?
4. Why configure static routes manually instead of relying on Mininet defaults?
5. What is `OVSBridge` vs. a learning switch, and why OVS in your topology?
6. How does `traffic/orchestrator.py` spawn iperf3 and ping from application registry profiles?
7. Why capture PCAP on `s_edge` and `s_core` switch interfaces?
8. What requires root privileges in your experiments, and how would you run CI without it?
9. How do you reproduce a 30-second school-hours experiment end-to-end?
10. What is the difference between your canonical topology and the legacy one-switch star topology?
11. How does `FaultInjector` simulate backhaul failure at the Linux link layer?
12. What limitations does Mininet impose on radio-layer modeling?
13. How would UERANSIM extend the architecture as a modular plugin?
14. What is the bottleneck link (`s_rural-eth4`) and why shape it at 100 Mbps?

8 Telemetry, Feature Extraction, and Dataset Design

1. What telemetry sources does `telemetry/collector.py` use? (`ovs-ofctl`, `tc`, PCAP, `psutil`)
2. Why is the network the source of truth for features, not the traffic generator?
3. Explain the 89-column schema and how `features/schema_mapper.py` maps partial PCAP data to it.
4. Which features are PCAP-derived, OVS-derived, TC-derived, context-injected, or spawn-metadata labels?
5. How are flow duration, IAT statistics, and throughput computed from PCAP timestamps?
6. What TCP flag counts are extracted and why do they matter for classification?

7. Why derive jitter from IAT standard deviation, and what are the limitations?
8. What is the RSNSD v1 synthetic dataset, and why is it frozen as bootstrap only?
9. Explain the baseline → Mininet → compare SliceNet training pipeline.
10. Why might synthetic F1 be 0.98 while Mininet F1 is much lower? Is that expected?
11. What is label leakage, and which columns were excluded from ML inputs?
12. How does time-aligned dataset building work in `dataset/builder.py`?
13. What validation score (out of 100) indicates a publishable Mininet-generated dataset?
14. Why use both CSV and optional Parquet export?
15. How would you extend feature extraction for encrypted traffic (no DPI)?

9 SliceNet and Traffic Classification

1. What is SliceNet's role in CARS-SF? What does it *not* do?
2. Classify flows into which four service classes? What are example applications per class?
3. Why flow-level classification instead of per-packet deep inspection?
4. What input features does `classification/slicenet.py` use after leakage exclusion?
5. How does inference integrate with the RL scheduler's observation vector?
6. What metrics do you report: macro F1, weighted F1, per-class F1, confusion matrix?
7. Why bootstrap on RSNSD v1 before retraining on Mininet data?
8. What is the difference between `slicenet_baseline.pkl` and `slicenet_mininet.pkl`?
9. How would a 1D-CNN on flow feature vectors differ from your RandomForest approach?
10. What happens when SliceNet misclassifies agriculture traffic as general?
11. How do you handle unknown or encrypted flows at inference time?
12. What is the latency budget for SliceNet inference on the edge server?
13. How would you update the model when new applications are added to the registry?
14. Why is classification accuracy alone insufficient for thesis claims?

10 Context Engine, Allocation, and QoS Enforcement

1. How does `slicing/context_engine.py` assign priority weights for normal, school, harvest, disaster?
2. What is the difference between priority weights and bandwidth allocation?
3. Explain the two-pass allocation in `slicing/allocation_engine.py` step by step.
4. What is surplus redistribution, and why a second pass?
5. How does `slicing/qos_enforcer.py` create HTB root qdisc and per-service classes?

6. What are u32 filters, and how do they map UE IPs to HTB class IDs?
7. What command applies 81.82 Mbps to healthcare class during disaster allocation?
8. Why might printed OVS commands (prototype mode) differ from real `tc` enforcement?
9. How does `--apply-qos` flag wire allocation output to the network?
10. What is the relationship between demand (Mbps), priority weight, and granted bandwidth?
11. How would weighted fair queuing differ from your proportional allocation approach?
12. What QoS metrics define SLA satisfaction per application in `registries/applications.yaml`?
13. How do you prevent starvation of general traffic while prioritizing healthcare?
14. What happens when total demand exceeds bottleneck capacity?

11 RL Scheduler and Control Loop

1. Why does the RL agent observe network state but not inspect raw packets?
2. List all observation dimensions in `scheduler/env.py` (context, demands, allocations, QoS).
3. What actions can the RL agent take on slice priorities and bandwidth caps?
4. How is the reward function defined? What is penalized?
5. What is `scheduler/baseline.py` and why must RL beat this heuristic?
6. Why use a Gym-compatible environment even without stable-baselines3?
7. What is the action space dimensionality, and how are actions normalized?
8. How does the RL agent differ from the rule-based context engine?
9. What is policy improvement in your closed loop, and what triggers it?
10. Why MultiOutputRegressor for action vectors instead of single-output regression?
11. How would PPO or DQN be integrated without changing the environment interface?
12. What are the risks of training RL on simulated QoS vs. live Mininet feedback?
13. How often should the RL agent reallocate (per tick, per second, on SLA violation)?
14. What is the exploration vs. exploitation trade-off during disaster scenarios?
15. How would you explain RL scheduler failure modes to a non-ML networking examiner?

12 Monitoring, Self-Healing, and Resilience

1. What does `monitoring/monitor.py` detect: latency, packet loss, queue overflow?
2. How are SLA thresholds tied to per-application `latency_requirement_ms`?
3. What failure signatures does `self_healing/detector.py` recognize?
4. How do you distinguish congestion-induced loss from fiber-cut-induced loss?
5. What recovery actions does `self_healing/recovery.py` plan?

6. Explain boost-priority vs. throttle-general vs. reroute-and-protect strategies.
7. How does `self_healing/injector.py` inject link down and latency (`netem`)?
8. Why must self-healing consume the same telemetry pipeline as SliceNet and RL?
9. What is the target: detection under 5s, automatic reroute, QoS restoration?
10. How would RL-based recovery extend rule-based recovery without merging modules?
11. What is the difference between failure detection and failure prediction?
12. How does monitoring integrate into `core/orchestrator.py` per tick?
13. What root cause analysis (RCA) labels does the v3 blueprint propose?
14. How would you test self-healing without destabilizing the entire Mininet experiment?

13 Core Systems and Software Engineering

1. Why Python for the control plane? What are latency implications on the edge?
2. How do you handle Mininet requiring root while ML training runs as normal user?
3. What is the single responsibility of each module per your engineering principles?
4. How do you avoid unnecessary coupling between RSNSD, MininetSims, and RSNSDv2?
5. Why `sys.path` inserts in experiment scripts, and what is the proper packaging fix?
6. How would you containerize the telemetry + SliceNet + scheduler stack?
7. What secrets or credentials must never appear in `registries/` or datasets?
8. How do you version datasets (`cars_sf_dataset.csv`, `mininet_full_dataset.csv`)?
9. What is reproducibility: fixed seeds, scenario YAML, duration, allocation bandwidth?
10. How would you add a fifth service class without rewriting the allocation engine?
11. What logging and observability would you add for production deployment?
12. How do you test modules that require Mininet without running full experiments?
13. What is the difference between orchestrator monolith and plugin-based pipeline?
14. How would you handle concurrent experiments on the same host?
15. What technical debt remains in the repository after architectural consolidation?

14 SDN, OpenFlow, and Linux Networking Internals

1. What is Open vSwitch, and how does it differ from Linux bridge?
2. What does `ovs-ofctl dump-flows` return, and how do you aggregate byte counts?
3. What is HTB (Hierarchical Token Bucket) and how does it differ from CBQ?
4. Explain `tc qdisc`, `tc class`, and `tc filter` hierarchy on an interface.
5. What is `netem` and when would you use it for failure injection?

6. How does iperf3 UDP vs. TCP traffic patterns differ for URLLC vs. eMBB emulation?
7. What is goodput vs. throughput, and how is goodput derived in your feature extractor?
8. How do queue drops in `tc -s class show` relate to packet loss rate features?
9. What is ARP timeout signature in fiber-cut failure detection?
10. Why static routes on multi-subnet Mininet topology (10.0.1.x, 10.0.2.x, 10.0.3.x)?
11. What would a Ryu/ONOS controller add that your current design lacks?
12. How does SDN enable dynamic rerouting in self-healing recovery?

15 Experimental Methodology and Evaluation

1. What experiments does `experiments/run_reproducible.py` run across scenarios?
2. How do you generate figures in `docs/figures/` for the thesis report?
3. What does `simulation_vs_synthetic.png` demonstrate?
4. What tables are in `docs/tables/` and what claims do they support?
5. How do you evaluate RL vs. rule-based scheduler quantitatively?
6. What is the success criterion for SliceNet on held-out Mininet flows (>85% F1)?
7. How many scenarios, applications, and flows constitute a sufficient Mininet dataset?
8. What ablation studies would strengthen the thesis (no context, no SliceNet, no RL)?
9. How do you report confidence in results given Mininet is not real 5G RAN?
10. What limitations section must you include for academic integrity?
11. How would field deployment on a real testbed validate Mininet findings?
12. What is the internship report vs. M.Tech thesis scope boundary in your repo?

16 Scenario-Based and Viva Questions

1. **Scenario:** School hours start at 8 AM with 80 Mbps education demand and only 100 Mbps backhaul. Walk through allocation and QoS enforcement.
2. **Scenario:** A fiber cut reduces OVS byte counters by 90% in one tick. What detects it, what recovers, and what metrics improve?
3. **Scenario:** SliceNet mislabels telemedicine as general during harvest season. What is the impact on allocation and patient SLA?
4. **Scenario:** An examiner says RSNSD v1 invalidates your "network is source of truth" claim. How do you respond?
5. **Scenario:** You must add Police/Emergency as a fifth slice with highest priority. What files change?
6. **Scenario:** PCAP capture fails mid-experiment. What does the pipeline produce, and is it scientifically valid?

7. **Scenario:** RL agent learns to starve general traffic entirely. Is that acceptable in disaster context?
8. **Scenario:** Compare your project to a standard 5G core network slicing implementation commercially.
9. **Scenario:** You have 10 minutes to demo the project live. What commands do you run and in what order?
10. **Scenario:** A journal asks for reproducibility. What do you publish besides code?

17 Quick-Fire Definitions (Oral Exam)

Service slice Logical end-to-end network partition with isolated resources and QoS.

Context-aware slicing Allocation driven by operational scenario, not static reservations.

SliceNet Flow-level classifier mapping traffic to rural service classes.

RL scheduler Agent that adjusts priorities/allocations from network state observations.

HTB Linux hierarchical token bucket scheduler for bandwidth shaping.

UPF User Plane Function; packet forwarding anchor in 5G core.

MEC Edge compute hosting low-latency applications near the RAN.

URLLC Ultra-reliable low-latency communications slice type.

Ground truth label Deterministic label from spawn metadata or measured SLA violation.

Label leakage Using oracle label information as ML input, inflating accuracy.

Closed loop Sense → decide → act → measure → learn cycle.

Digital twin Mininet emulation mirroring production network topology and traffic.

Surplus redistribution Reallocating unused bandwidth to still-starved slices.

Markovian state RL observation containing sufficient information for optimal decisions.

Zero-synthetic policy Features must originate from measurable network behaviour.

18 Repository Map (Answer Key Reference)

Suggested preparation workflow:

1. Answer each section without looking at code (30 min per section).
2. Cross-check answers against repository files and `ARCHITECTURE.md`.
3. Practice drawing the pipeline and rural topology from memory.
4. Run `demo.py`, `run_slicenet_pipeline.py --skip-collect`, and explain outputs live.
5. Prepare 2-minute and 10-minute project pitches using Sections 1 and 2.

Module	**Path**
Orchestrator	`core/orchestrator.py`
Application registry	`registries/applications.yaml`
Scenario registry	`registries/scenarios.yaml`
Rural topology	`topology/rural_topology.py`
Traffic spawn	`traffic/orchestrator.py`
Telemetry	`telemetry/collector.py`
Feature extraction	`features/extractor.py`, `features/schema_mapper.py`
SliceNet	`classification/slicenet.py`, `classification/pipeline.py`
RL scheduler	`scheduler/agent.py`, `scheduler/env.py`
Allocation	`slicing/allocation_engine.py`
QoS enforcement	`slicing/qos_enforcer.py`
Monitoring	`monitoring/monitor.py`
Self-healing	`self_healing/recovery.py`
Mininet entry	`run_experiment.py`
SliceNet pipeline	`experiments/run_slicenet_pipeline.py`

Table 1: Key repository paths for interview cross-reference.